



Universidad
Zaragoza

Trabajo Fin de Grado

Inteligencia Artificial On-Premise para el análisis de
IoCs con datos privados

Autor

Nicolás Pueyo Soria

Directores

Héctor Francia Molinero

Ricardo Julio Rodríguez Fernández

ESCUELA DE INGENIERÍA Y ARQUITECTURA
2026

Agradecimientos

Resumen

Aquí irá el abstract cuando acabe de escribir

Abstract

In English ig

Índice

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Objetivos del proyecto	1
1.3. Estructura del documento	2
2. Conceptos previos	3
2.1. IR e IoCs	3
2.2. Ciberinteligencia de Amenazas (CTI) y OpenCTI	5
2.3. Modelos de Lenguaje de Gran Escala (LLM)	5
2.4. Motores de inferencia local	5
2.5. Generación Aumentada por Recuperación (RAG)	6
3. Análisis, diseño e implementación del sistema	7
3.1. Arquitectura del sistema	10
3.2. Workflow del sistema	11
3.2.1. Consumo y vectorización de OpenCTI	12
3.2.1.1. Ingestión	12
3.2.1.2. Normalización	13
3.2.1.3. Generación de <i>embeddings</i> e inserción en <i>ChromaDB</i>	15
3.2.2. Frontend, backend y motor de RAG	16
3.2.2.1. OpenWebUI	16
3.2.2.2. Servidor <i>Pipelines</i>	16
3.2.2.3. Backend y motor RAG	16
4. Evaluación y discusión de resultados	17
5. Conclusiones y trabajo futuro	18
Anexos	19
Anexo A. Organización del trabajo.	19
Anexo B. Ficheros desarrollados.	21
Bibliografía	32

1. Introducción

1.1. Contexto y motivación

En los últimos años, la adopción de tecnologías basadas en Inteligencia Artificial (IA) se ha consolidado como una práctica habitual en el entorno corporativo para optimizar procesos y mejorar la productividad de los empleados. En particular, los Modelos de Lenguaje de Gran Escala (LLM) han demostrado ser herramientas de utilidad para tareas como la síntesis de información y la redacción de informes técnicos. No obstante, este rápido despliegue tecnológico también introduce retos de seguridad significativos. La falta de formación específica en ciberseguridad de muchos usuarios puede dar lugar a malas prácticas en el manejo de la IA, como la exposición accidental de datos corporativos sensibles al enviarlos a APIs comerciales externas.

De forma paralela, la defensa frente a incidentes digitales se apoya cada vez más en la Ciberinteligencia de Amenazas (CTI, por sus siglas en inglés), la cual permite estructurar y correlacionar datos sobre potenciales ataques mediante herramientas de código abierto como OpenCTI. Sin embargo, la gestión diaria de estos repositorios de conocimiento sigue requiriendo una considerable inversión de tiempo por parte de los analistas para procesar e interpretar la información técnica de los nuevos Indicadores de Compromiso (IoC).

En este escenario, surge la oportunidad de integrar la IA generativa como un asistente para facilitar la labor de los analistas en la interpretación y análisis de CTI. Para llevar a cabo esta integración en un contexto empresarial de manera segura, es necesario descartar las soluciones basadas en la nube y optar por modelos locales y privados. De este modo, este proyecto propone el desarrollo de una herramienta desplegada en un servidor privado que, mediante modelos de lenguaje especializados ejecutados localmente, permita a los ingenieros de ciberseguridad consultar, comparar y contrastar la ciberinteligencia de la organización con nuevos indicadores en un tiempo razonable, garantizando la soberanía de los datos sensibles de la empresa.

El presente proyecto se ha desarrollado en el marco de colaboración con la división de Zaragoza de Inetum España S.A., multinacional especializada en consultoría tecnológica y provisión de servicios de TI. En Aragón, Inetum desempeña un papel relevante al gestionar las infraestructuras y los servicios informáticos de entidades públicas de gran envergadura, como la Diputación General de Aragón (DGA) y el Servicio de Salud Aragonés (SALUD). En este contexto empresarial, la infraestructura asignada para el despliegue de la solución consta de entornos de virtualización Linux (RedHat 10), sin entorno de escritorio, sobre hardware de centro de datos basados únicamente en capacidad de procesamiento (CPU) y memoria RAM, prescindiendo por completo de aceleración gráfica por GPU. Además, con el fin de garantizar la soberanía, la transparencia y la viabilidad de la arquitectura propuesta, el sistema se implementará de manera íntegra utilizando herramientas y soluciones de software de código abierto.

1.2. Objetivos del proyecto

El objetivo general de este proyecto es diseñar, implementar y evaluar una plataforma privada y local de asistencia conversacional orientada a la ingeniería de ciberseguridad, integrada con una plataforma de gestión de ciberinteligencia de amenazas (CTI). El

sistema propuesto debe permitir la consulta interactiva y el análisis contextualizado de Indicadores de Compromiso (IoC) garantizando la soberanía de los datos sensibles y optimizando el rendimiento de modelos de lenguaje bajo restricciones estrictas de hardware basadas exclusivamente en procesamiento por CPU y memoria RAM.

Para alcanzar este fin, se proponen los siguientes objetivos específicos:

- **Despliegue y configuración de un entorno local aislado:** Implementar una infraestructura privada que albergue un entorno de ejecución para modelos de lenguaje y una interfaz gráfica de usuario accesible para los analistas, operando de manera local y sin dependencias de servicios externos en la nube.
- **Desarrollo de un pipeline de recuperación semántica:** Diseñar un mecanismo de indexación, almacenamiento vectorial y recuperación semántica de datos que permita alimentar las consultas del asistente conversacional con la ciberinteligencia y el contexto histórico de la organización de forma precisa.
- **Integración y consulta bidireccional con sistemas de ciberinteligencia:** Establecer una conexión con la API de la plataforma de ciberinteligencia del cliente para permitir que el asistente interactúe con el grafo de conocimiento existente, traduciendo consultas en lenguaje natural a búsquedas estructuradas de observables y amenazas.
- **Evaluación comparativa de modelos de lenguaje especializados:** Evaluar el rendimiento, el nivel de detalle técnico, la tasa de alucinaciones y la precisión de múltiples modelos de lenguaje de diferentes escalas de parámetros en tareas complejas de correlación y análisis de indicadores.

1.3. Estructura del documento

El presente documento se estructura en diferentes secciones con subsecciones. En la segunda sección se explican los conocimientos teóricos previos a tener en cuenta sobre diferentes conceptos clave para entender el desarrollo del proyecto. En la tercera sección se detalla cuál es la arquitectura del proyecto, qué componentes se han utilizado, qué tecnologías se han usado para desarrollar o desplegar cada uno de estos componentes, así como qué organización tienen y cómo se despliegan físicamente cada uno, y finalmente explica el workflow general del sistema, es decir, cómo cumple cada componente su cometido. En la cuarta sección se detallan las pruebas que se han realizado al sistema, se recogen y presentan los resultados obtenidos y se realiza un análisis de los mismos. Finalmente, en la última sección, la quinta, se analiza todo el contexto y trabajo desarrollado para extraer unas conclusiones, y se reflexiona sobre qué otras funcionalidades se podrían añadir al proyecto en caso de continuar con el desarrollo.

2. Conceptos previos

2.1. IR e IoCs

En el ámbito de la ciberseguridad corporativa, la Respuesta a Incidentes (IR, por sus siglas en inglés de *Incident Response*) constituye la metodología estructurada y sistemática que adopta una organización para la detección, contención, mitigación y recuperación frente a incidentes de seguridad y ciberataques. De acuerdo con los estándares de la industria, el objetivo primordial de la respuesta a incidentes abarca tanto la prevención proactiva de ciberataques antes de que estos lleguen a materializarse, como la minimización del impacto financiero, el daño reputacional y la interrupción de los procesos de negocio en caso de que una intrusión se complete de manera exitosa.

Para operativizar esta estrategia, las organizaciones definen sus tecnologías, responsabilidades y procedimientos dentro de un Plan de Respuesta a Incidentes (IRP, *Incident Response Plan*). Este plan actúa como el componente técnico de la gestión integral de incidentes, especificando cómo deben identificarse, aislarse y resolverse los diferentes vectores de ataque de forma coordinada. En la actualidad, la integración de soluciones de ciberseguridad asistidas por inteligencia artificial permite acelerar este proceso drásticamente, ayudando a los analistas a predecir posibles canales de ataque, automatizar la clasificación de alertas y correlacionar comportamientos sospechosos antes de que escalen a incidentes críticos.

Para el estudio del estado de la ciberinteligencia en el marco de la respuesta a incidentes, los analistas dependen de los Indicadores de Compromiso (IoCs, *Indicators of Compromise*). En el campo de la informática forense y el análisis de malware, un IoC se define como cualquier artefacto, registro o evidencia digital identificado en un sistema operativo, un dispositivo de red o un flujo de tráfico que permite constatar, con un determinado nivel de confianza, la presencia de una actividad maliciosa o el compromiso de un activo.

En el marco del estándar internacional STIX 2.1, la arquitectura de datos sobre la que se fundamenta la plataforma OpenCTI utilizada en este proyecto, es imperativo establecer una distinción técnica entre un *Observable* (SCO, *STIX Cyber Observable*) y un *Indicador* (SDO, *STIX Domain Object*):

- **Observable (SCO):** Representa un dato técnico en bruto desprovisto de intencionalidad o valoración ética. Una dirección IP, un dominio o el hash de un binario son observables que pueden pertenecer tanto a servicios legítimos de la infraestructura como a servidores del atacante.
- **Indicador (SDO):** Es la entidad que aporta contexto semántico y de seguridad al observable. Un indicador asocia un patrón de búsqueda (definido bajo estándares como STIX Pattern, reglas YARA o firmas Sigma) a un observable específico, determinando un período de validez temporal, relaciones con campañas de actores de amenazas y un nivel de certeza técnica de maliciosidad.

Para el alcance del sistema desarrollado en este proyecto, se considerarán y analizarán los siguientes tipos de observables e indicadores de compromiso técnicos:

- **URLs** identificadas como maliciosas, habitualmente vinculadas a servidores de descarga de cargas útiles (*payloads*), portales de suplantación de identidad

(*phishing*) o paneles de control de botnets.

- **Hashes** de ficheros identificados como códigos dañinos. El uso de funciones de dispersión criptográfica permite identificar unívocamente muestras de malware sin necesidad de procesar el archivo completo. Para este proyecto se contemplan:
 - **SHA256** (64 caracteres hexadecimales), estándar actual por su resistencia a colisiones.
 - **SHA1** (40 caracteres hexadecimales), de uso remanente en bases de datos históricas.
 - **MD5** (32 caracteres hexadecimales), obsoleto criptográficamente pero común en la categorización rápida de amenazas.
- **Direcciones IPv4** identificadas como maliciosas, asociadas comúnmente a infraestructura de Comando y Control (C2), nodos de salida Tor o proxies utilizados para el escaneo de vulnerabilidades.
- **Dominios** web identificados como maliciosos, utilizados para enmascarar direcciones IP de infraestructura atacante mediante técnicas de resolución dinámica de nombres.

A nivel estratégico, la utilidad de estos IoCs se rige bajo la *Pirámide del Dolor* de David Bianco[1]. Este modelo conceptual organiza los indicadores de amenazas según el grado de dificultad (o "dolor") que causa al atacante el hecho de que un defensor los detecte y bloquee:

1. **Base (Hashes, IPs y Dominios):** Constituyen el nivel más bajo. Son extremadamente sencillos y baratos de modificar para el adversario. Un atacante puede alterar un solo byte de su código para cambiar el hash criptográfico del malware, o rotar de dirección IP en segundos mediante servicios de red dinámica, eludiendo bloqueos automatizados estáticos.
2. **Medio (Artefactos de red y de host):** Requiere que el atacante modifique los patrones de comunicación de sus herramientas o las trazas que estas dejan en los registros de los sistemas comprometidos.
3. **Cúspide (Herramientas y TTPs):** Las Tácticas, Técnicas y Procedimientos (TTPs) definen el comportamiento operativo real y los hábitos del atacante. Obligar a un atacante a cambiar sus TTPs es el logro de mayor valor defensivo, ya que le exige reestructurar por completo su metodología y reaprender tácticas de infiltración.

La integración de la Inteligencia Artificial propuesta en este trabajo encuentra su motivación en esta jerarquía. Mientras que los sistemas automáticos de seguridad tradicionales (como cortafuegos o antivirus) son excelentes procesando de manera rígida los observables de la base (hashes e IPs), carecen de la capacidad cognitiva para relacionar estos datos con comportamientos complejos. La incorporación de un modelo de lenguaje de gran escala (LLM) permite a los ingenieros de ciberseguridad procesar el texto no estructurado y los grafos de conocimiento de OpenCTI para correlacionar observables simples con el comportamiento general de la amenaza en la cúspide de la pirámide, forzando un impacto real sobre el coste operativo del atacante.

2.2. Ciberinteligencia de Amenazas (CTI) y OpenCTI

La Ciberinteligencia de Amenazas (CTI, por sus siglas en inglés de *Cyber Threat Intelligence*) consiste en la recopilación, análisis y estructuración de información sobre amenazas activas o potenciales con el fin de anticipar, detectar y mitigar ataques de forma proactiva. Esta disciplina permite a los equipos de seguridad comprender el contexto, las motivaciones y las metodologías de los adversarios, optimizando la toma de decisiones estratégicas y los tiempos de respuesta ante incidentes.

Para centralizar y procesar este conocimiento, las organizaciones emplean plataformas especializadas (TIP, *Threat Intelligence Platforms*) como OpenCTI, una herramienta de código abierto diseñada para estructurar, almacenar, organizar y visualizar información técnica y no técnica sobre ciberamenazas. El núcleo de OpenCTI se fundamenta en un grafo de conocimiento basado en el estándar internacional STIX 2.1. En esta arquitectura, la inteligencia se modela mediante dos elementos[2]:

- **Nodos (Entidades):** Representan conceptos de alto nivel, conocidos como objetos de dominio o *STIX Domain Objects* (SDO), tales como Malware, Actores de Amenazas o Vulnerabilidades.
- **Aristas (Relaciones):** Modelan las conexiones lógicas entre las entidades y se asocian a observables técnicos o *STIX Cyber Observables* (SCO), como direcciones IP, dominios o hashes de archivos, que sirven de evidencia de la actividad.

2.3. Modelos de Lenguaje de Gran Escala (LLM)

Los Modelos de Lenguaje de Gran Escala (LLM, por sus siglas en inglés) son sistemas basados en redes neuronales profundas diseñados para comprender, procesar y generar lenguaje natural. Formalmente, un LLM se define como una función matemática compleja cuya entrada y salida consisten en listas de números enteros[3]. Para que el modelo pueda procesar el texto en lenguaje humano, este debe someterse a un proceso de tokenización, en el cual se divide el flujo de caracteres en unidades discretas menores llamadas tokens (que pueden ser palabras, subpalabras o caracteres individuales)[4]. Este proceso mapea el texto a valores enteros dentro de un rango determinado por el tamaño de su vocabulario $[0, V - 1]$, donde V representa la dimensión del vocabulario del modelo[3].

En el contexto corporativo, la adopción de estos modelos se divide principalmente entre servicios de infraestructura gestionada en la nube y despliegues locales. Si bien el uso de APIs comerciales en plataformas en la nube (como las soluciones de infraestructura de AWS) proporciona una gran capacidad de cómputo y escalabilidad, la transferencia de datos de seguridad hacia servidores externos introduce riesgos severos de exposición, fugas de información y fallas de cumplimiento normativo. Por esta razón, el estado del arte en ciberseguridad se orienta firmemente hacia el despliegue local de modelos de pesos abiertos de menor tamaño, garantizando la soberanía absoluta de los datos sensibles de la organización[5].

2.4. Motores de inferencia local

Mientras que un LLM representa la estructura estática de la red neuronal y sus pesos matemáticos almacenados en disco, el motor de inferencia es el software de ejecución

(*runtime*) activo que carga dichos pesos en memoria y realiza las operaciones algebraicas necesarias para responder a las consultas del usuario. En un servidor privado que carece de aceleradores gráficos (GPU), el motor de inferencia debe ejecutar el modelo utilizando exclusivamente el procesador (vCPU en contextos de centros de datos) y la memoria RAM del sistema.

En este escenario de inferencia puramente basado en CPU, la velocidad del sistema se ve fuertemente limitada. El cuello de botella principal no radica en la capacidad de cómputo de la vCPU, sino en el ancho de banda físico de la memoria RAM, puesto que el procesador debe leer la totalidad de los parámetros del modelo para generar cada token individual. Mientras que las GPUs utilizan memoria especializada de gran velocidad con un ancho de banda de entre 600 y 1500 GB/s, la memoria RAM convencional de un servidor host suele transferir datos a velocidades de entre 50 y 200 GB/s.

Esta limitación física provoca que la asignación de hilos de ejecución en el motor de inferencia no escale de manera lineal con el número de núcleos virtuales (vCPUs) asignados. Si se configuran demasiados hilos de ejecución concurrentes en el motor de inferencia para procesar un modelo pesado, los hilos terminarán compitiendo por los mismos canales físicos de acceso a la memoria RAM. Esta contención de memoria genera un cuello de botella técnico que degrada drásticamente el rendimiento del sistema y reduce la tasa de generación de texto (tokens por segundo), por lo que la optimización de estos sistemas requiere limitar de forma explícita el número de hilos de ejecución a un valor cercano al número de núcleos físicos del procesador.

2.5. Generación Aumentada por Recuperación (RAG)

La Generación Aumentada por Recuperación (RAG, por sus siglas en inglés de *Retrieval-Augmented Generation*) es una técnica que optimiza el comportamiento de un modelo de lenguaje al complementar sus instrucciones (*prompts*) de entrada con información externa relevante sin necesidad de modificar físicamente los parámetros del modelo[4]. El flujo operativo de RAG consta de tres etapas lógicas:

1. **Recuperación:** Ante una consulta del analista, el sistema busca de manera automática los fragmentos de documentos o informes técnicos más relevantes dentro de una base de datos.
2. **Aumentación:** El sistema añade estos fragmentos recuperados al *prompt* original del usuario en forma de contexto informativo verídico.
3. **Generación:** El LLM procesa la consulta enriquecida y genera una respuesta redactada en lenguaje natural que se fundamenta estrictamente en la evidencia proporcionada, reduciendo de manera drástica las alucinaciones del sistema[4].

Para dar soporte y agilizar la etapa de recuperación en tiempo real, se emplean las bases de datos vectoriales (como ChromaDB o Qdrant), las cuales almacenan los textos de ciberinteligencia previamente convertidos en vectores numéricos o *embeddings* que capturan su significado semántico. De este modo, la base de datos es capaz de localizar de forma matemática los fragmentos de información más pertinentes según la intención de la consulta del analista, inyectándolos instantáneamente en el contexto de la IA[3].

3. Análisis, diseño e implementación del sistema

Tomando como punto de partida la infraestructura tecnológica preexistente de la organización, la cual dispone de una instancia de OpenCTI desplegada de manera independiente en un nodo dedicado de la red corporativa, el desarrollo práctico de este trabajo se centra en diseñar e implementar una pasarela de comunicación bidireccional y segura con dicha base de conocimiento.

Con el propósito de estructurar de forma modular el asistente inteligente, cuya finalidad es procesar y correlacionar información de seguridad dentro del perímetro de la empresa, la arquitectura del sistema propuesto se articula en torno a los siguientes componentes:

- **Motor de inferencia local y selección de modelos:** Para la ejecución de los modelos de lenguaje de gran escala bajo un entorno local, se evaluaron diversas alternativas de motores de inferencia, entre las que destacaron Ollama y vLLM. Si bien vLLM ofrece un rendimiento excepcional en la gestión de solicitudes concurrentes gracias al algoritmo de optimización de memoria *PagedAttention* [6], su arquitectura está diseñada de manera prioritaria para operar bajo aceleración por hardware gráfico (GPU). Debido a la restricción de infraestructura impuesta en este proyecto, consistente en operar exclusivamente sobre procesamiento por CPU, se seleccionó Ollama como el motor de inferencia local principal. Ollama, desarrollado sobre la biblioteca nativa en C/C++ llama.cpp, proporciona una capa de abstracción eficiente para entornos virtualizados, facilidad de configuración multimodelo mediante archivos de instrucción (*Modelfiles*) y una API local compatible con los estándares de la industria.

Para posibilitar la interacción con el motor, los modelos de lenguaje se cargan en su versión cuantizada utilizando archivos de extensión `.gguf`. En cuanto a los modelos de lenguaje seleccionados, se descartaron las opciones generalistas en favor de modelos especializados en ciberseguridad mediante ajuste fino (*fine-tuning*). Esta especialización garantiza que la IA sea capaz de interpretar terminología técnica de ciberinteligencia, correlacionar observables y procesar de manera segura datos potencialmente ofensivos (como volcados de registros o firmas de malware) sin sufrir de bloqueos por políticas de seguridad corporativas comerciales. Con el fin de realizar una evaluación comparativa bajo los recursos asignados al servidor, se seleccionaron dos modelos especializados de escalas de parámetros diferenciadas:

- **Lily-Cybersecurity-7B-v0.2:** Modelo ligero desarrollado por Sego Lily Labs, ajustado sobre la arquitectura de Mistral-7B-Instruct-v0.2 a partir de un conjunto de 22,000 instrucciones de seguridad manuales [7]. En su cuantización a 4 bits, presenta un peso de archivo de aproximadamente 4.37 GB, lo que permite una inferencia ágil y fluida con un consumo mínimo de memoria RAM.
- **CyberOSS-2.0-20B (CyberPal-2.0-20B):** Modelo avanzado de la suite de seguridad de código abierto CyberPal 2.0, ajustado sobre la arquitectura gpt-oss-20b mediante el pipeline de enriquecimiento y trazado de razonamiento técnico *SecKnowledge 2.0* [8]. En su cuantización de 4 bits (Q4_K_M), el tamaño del archivo asciende a 15.8 GB, demandando un mayor esfuerzo de lectura por parte de la memoria RAM del servidor, pero aportando un análisis conceptual de nivel experto.

La elección de esta pareja de modelos permite someter la herramienta a una validación práctica frente a los benchmarks de referencia de la literatura científica. De acuerdo con el estudio de evaluación integral publicado en "Toward Cybersecurity-Expert Small Language Models" (arXiv:2510.14113) [8], Lily-Cybersecurity-7B-v0.2 obtiene una puntuación promedio de 53.38 % en la resolución de tareas complejas de seguridad y ciberinteligencia, mientras que CyberPal-2.0-20B alcanza una puntuación de 80.33 % en el mismo conjunto de pruebas (CTI-Bench), superando incluso a modelos comerciales cerrados en la correlación de vulnerabilidades e identificación de tácticas.

- **Interfaz gráfica de usuario y orquestación de flujos (Open WebUI):** Para proporcionar a los analistas de ciberseguridad un entorno visual, accesible e interactivo, se evaluaron diversas plataformas autohospedadas capaces de desplegarse mediante contenedores Docker y configurarse de manera sencilla a través de parámetros de entorno. En una primera fase de diseño, se contempló la implementación de la interfaz *ChatUI* desarrollada por Hugging Face. Sin embargo, este entorno fue descartado debido a incompatibilidades severas en su integración nativa con el motor de inferencia local Ollama. Durante las pruebas de concepto, la ambigüedad en la documentación oficial respecto a su interconexión local propició fallos críticos en el control del estado, tales como la imposibilidad de mapear y listar dinámicamente los modelos de Ollama, y respuestas de los modelos no mostradas en la interfaz.

Para solucionar estos inconvenientes, se seleccionó la plataforma *Open WebUI*. Esta herramienta destaca por su compatibilidad nativa e inmediata con la API de Ollama, configuración de parámetros tanto de forma declarativa (en la configuración de Docker) como en caliente desde su panel de administración web [9]. Un factor tecnológico determinante para su elección fue la disponibilidad de su arquitectura desacoplada de flujos de trabajo, denominada *Pipelines*. Este servidor satélite programado en Python actúa como un *middleware* capaz de interceptar, filtrar y modificar las peticiones dirigidas al modelo de lenguaje [10]. Esta capacidad resulta crítica para el alcance de este proyecto, ya que proporciona el punto de anclaje necesario para inyectar lógica personalizada, realizar llamadas a servicios externos y canalizar las búsquedas de ciberinteligencia dentro de la base de datos vectorial de forma transparente para el analista, facilitando así la integración del sistema RAG.

- **Backend, motor de RAG y pipelines:** Este bloque representa el núcleo diferencial y el valor de desarrollo propio de este proyecto. Mientras que los componentes analizados hasta ahora se basan en el despliegue, parametrización y organización arquitectónica de imágenes de Docker (como Open WebUI o la base de datos vectorial), es en esta capa donde se introduce la lógica personalizada que gobierna el flujo, tratamiento y enriquecimiento de los datos de seguridad. Aunque se describen como componentes diferenciados, el servidor de *pipelines*, el backend de servicios y el motor de RAG operan como un único subsistema lógicamente acoplado debido a su constante cooperación en tiempo real.

El flujo de procesamiento de la información se inicia cuando el analista realiza una consulta en la interfaz gráfica que menciona un indicador de compromiso (IoC), momento en el cual la solicitud es interceptada de manera inmediata por

el servidor de *middleware Pipelines*. En lugar de transmitir la petición de forma directa al motor de inferencia local, el *pipeline* extrae el *payload* y desvía la solicitud hacia un *endpoint* específico de nuestro backend. Al recibir este paquete, el backend analiza la petición y llama a las funciones del motor de RAG, el cual recupera contexto técnico sobre el indicador consultando de manera estructurada tanto la API de OpenCTI como la base de datos vectorial local (ChromaDB). Una vez obtenido este contexto histórico y relacional, se devuelve en forma de respuesta al *pipeline*, encargado de reformular el *prompt* del usuario inyectando la información recuperada y de transmitir la instrucción final enriquecida al modelo correspondiente para que este realice su razonamiento.

Para la construcción del backend de servicios se seleccionó el marco de trabajo (*framework*) FastAPI. Esta elección se fundamenta principalmente en su sencillez y en que permite un desarrollo de *endpoints* de manera rápida y eficiente, facilitando un acoplamiento ágil con el resto de componentes escritos en Python. Asimismo, FastAPI destaca por su capacidad para gestionar de manera óptima la asincronía de forma nativa a través del servidor web de aplicaciones de bajo nivel Uvicorn, encargado de procesar la especificación ASGI de manera eficiente. Como un añadido a esta infraestructura, FastAPI utiliza internamente Starlette para dar soporte a la asincronía y se integra de forma directa con Pydantic para la validación declarativa de datos mediante anotaciones de tipo, lo que garantiza la integridad de los *payloads* intercambiados sin penalizar la velocidad de desarrollo.

- **Motor de base de datos vectorial (ChromaDB):** La incorporación de este componente responde a la necesidad de complementar la información relacional y estructurada obtenida de la API de OpenCTI con capacidades de recuperación semántica profunda. Mediante el uso de un modelo local de generación de vectores (*embeddings*), el sistema procesa, fragmenta e indexa tanto los informes técnicos de amenazas como los metadatos de los observables extraídos de la plataforma de ciberinteligencia. De este modo, ante una consulta del analista, la base de datos permite ejecutar búsquedas de similitud para deducir inferencias latentes y proporcionar un contexto complementario de alto valor que no se encuentra explícitamente enlazado en el grafo de conocimiento original.

Para la implementación de este almacén de vectores se seleccionó *ChromaDB*. En primer lugar, se define como una base de datos vectorial de código abierto altamente ligera y optimizada para despliegues locales (*on-premise*), lo que evita introducir una sobrecarga de infraestructura compleja o costosa en el servidor. En segundo lugar, su arquitectura modular permite que la base de datos se ejecute de forma embebida directamente dentro del proceso de Python o de manera independiente mediante un contenedor, lo que optimiza drásticamente el uso de recursos y memoria RAM en entornos limitados a CPU. Finalmente, destaca su excelente compatibilidad con el ecosistema de desarrollo de Python y su amplia adopción en la comunidad, lo que garantiza una API sencilla para operaciones de inserción, consulta, actualización y borrado (CRUD), así como un soporte nativo para el filtrado avanzado de metadatos.

- **Módulo de comunicación, ingesta y normalización de ciberinteligencia:** Este componente representa la capa de software desarrollada a medida para interactuar con la plataforma OpenCTI, resolviendo la adquisición de datos de

seguridad y su transformación hacia la base de datos vectorial. La conectividad lógica con el servidor se implementa bajo el patrón de diseño *Singleton* con soporte multihilo (*thread-safe*) mediante un mecanismo de exclusión mutua (*lock*), garantizando que exista una única instancia activa del cliente de API oficial (`OpenCTIApiClient`) durante la ejecución del sistema, lo que evita la saturación de conexiones GraphQL en el nodo de OpenCTI.

Sobre esta infraestructura de conexión se articulan tres subcomponentes de desarrollo propio en Python:

- **Cliente de consultas semánticas (`OpenCTIRAGClient`):** Componente que se ejecuta en tiempo real dentro del flujo de la conversación. Permite realizar búsquedas exactas e intermedias sobre observables e indicadores bajo la estructura del estándar STIX 2.1, rastreando de forma automática las relaciones lógicas (*relationships*) creadas por los analistas y localizando reportes técnicos relevantes que sirvan como evidencia contextualizada para el modelo.
- **Ingestor de entidades (`OpenCTIIngestor`):** Módulo encargado del aprovisionamiento masivo de datos históricos. Implementa el protocolo de paginación oficial basado en cursores de OpenCTI (`withPagination=True` y `endCursor`) para descargar de forma secuencial y ordenada colecciones completas de actores de amenazas, malware, tácticas, reportes e indicadores técnicos, almacenando la información de ciberinteligencia en bruto localmente en formato JSON.
- **Normalizador semántico e indexador (`OpenCTINormalizer`):** Clase de vital importancia para el RAG que resuelve el grafo de relaciones de OpenCTI (como la arista *uses* que vincula a un actor de amenazas con un malware específico o un TTP de MITRE ATT&CK). El normalizador extrae las relaciones complejas y las unifica en plantillas textuales estructuradas y metadatos legibles para la IA. Posteriormente, un script de vectorización automatizado codifica estos textos normalizados en representaciones densas de 384 dimensiones empleando el modelo local `all-MiniLM-L6-v2`, insertándolos finalmente mediante un algoritmo por lotes (*batches*) en ChromaDB bajo una métrica de similitud de distancia del coseno para dar soporte a las búsquedas de proximidad semántica.

3.1. Arquitectura del sistema

La arquitectura de la solución propuesta se ha diseñado siguiendo un paradigma modular y desacoplado. Por restricciones corporativas, sólo se dispone de una máquina virtual, por lo que todo está desplegado en la misma. No obstante, el sistema está diseñado para ser fácilmente distribuido. Esta topología, detallada en el diagrama de componentes de la Figura 1, permite separar de forma estricta el plano de interacción con el usuario e inferencia local del plano de ingesta, normalización y ciberinteligencia de amenazas.

A nivel de infraestructura, la comunicación entre los componentes se realiza a través de puertos y endpoints específicos dentro de una red virtual interna, articulándose en dos bloques tecnológicos principales:

- **Dominio de Interacción e Inferencia Local (Plano Izquierdo):** Este dominio gestiona la interfaz del analista y la ejecución de la IA. El componente *OpenWebUI* actúa como el frontend del sistema, comunicándose directamente con el puerto 9099 del *Pipeline Server*. Este último actúa como un *middleware* de interceptación y enrutamiento, derivando las consultas de enriquecimiento hacia el *Backend RAG* en el puerto 9000, concretamente al endpoint `/query-context` y transmitiendo el prompt final aumentado hacia el endpoint `/api/chat` de *Ollama* en el puerto 11434, el cual expone localmente los modelos *Lily* y *CyberOSS*.
- **Dominio de Ciberinteligencia, Ingesta y Vectorización (Plano Derecho):** Este dominio administra la adquisición de conocimiento de seguridad. El componente *OpenCTI Client* encapsula la conexión única hacia el puerto 8080 de la instancia externa de *OpenCTI*. De este cliente dependen de manera exclusiva el *Ingestor* (responsable del aprovisionamiento masivo de SDOs y SCOs) y el *Backend RAG* (responsable de las consultas semánticas y relacionales en caliente). Ambos módulos escriben y consultan, respectivamente, la base de datos vectorial *ChromaDB* a través de su API REST en el puerto 8080.

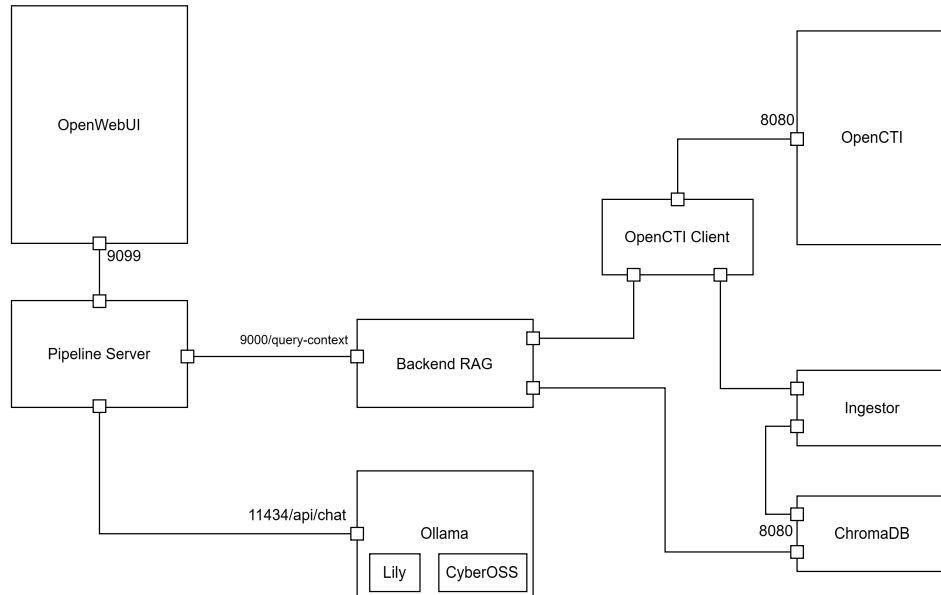


Figura 1: Diagrama de componentes

3.2. Workflow del sistema

En la Figura 2 se detalla concretamente la organización del despliegue de todos los artefactos mencionados, así como las comunicaciones que tienen entre ellos. Las comunicaciones se expresan como flechas etiquetadas. Una flecha bidireccional ($\longleftrightarrow$) significa que se manda una *request* y se espera *response* (por ejemplo, la flecha etiquetada con 1 entre *OpenWebUI* y *Pipelines*, mientras que una flecha en una sola dirección ($\longrightarrow$) significa que simplemente se envían datos en una sola dirección (esto solo se da entre el *ingestor* y *ChromaDB*, etiquetada con II).

Una vez definida la distribución lógica y física de la infraestructura, es fundamental definir la dinámica temporal que sigue el sistema. Como se ha mencionado previamente, el sistema sigue dos flujos de datos diferentes, por una parte está la ingesta, vectorización

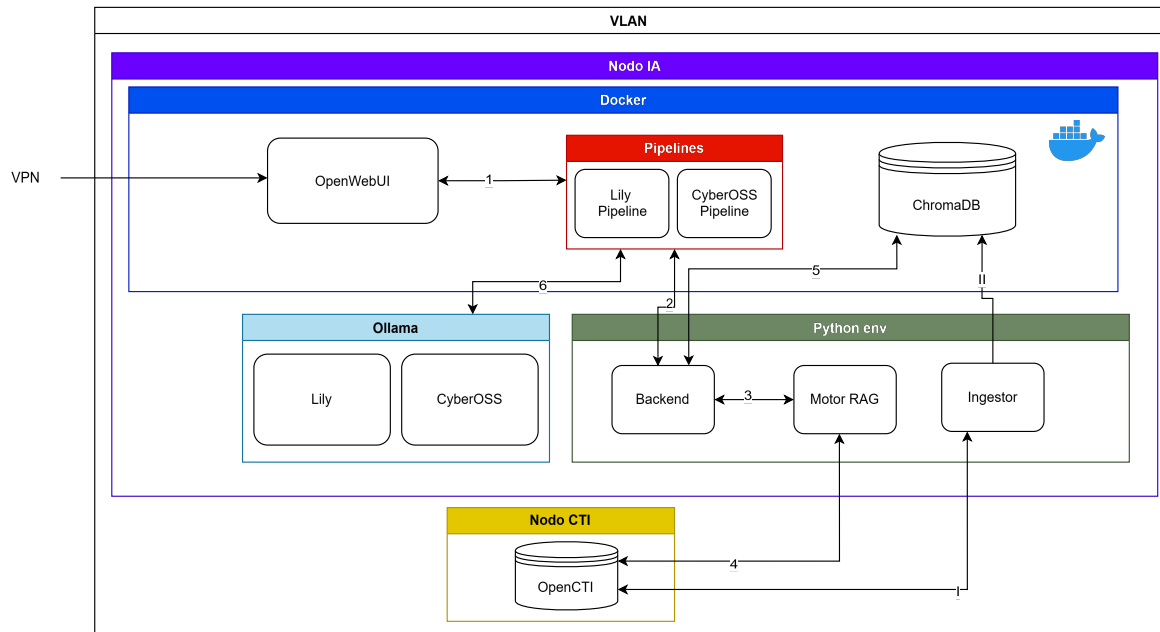


Figura 2: Diagrama de despliegue con comunicaciones

y almacenamiento de datos de OpenCTI (módulo *Ingestor* que interactúa con *OpenCTI* y *ChromaDB*), cuyas comunicaciones están etiquetadas con números romanos en el la figura

Finalmente, en la Figura 3 se detalla dentro del propio **Nodo IA**, la distribución de los directorios y los ficheros desarrollados para llevar a cabo el proyecto. Como se observa en la disposición jerárquica de la ilustración, se separan las funciones, por ejemplo, los despliegues de *Docker* se centralizan en un fichero `docker-compose.yml` en el directorio `chat-feats`, los modelos de IA para definir en Ollama se guardan en el directorio `modelos_ia`, los pipelines tienen su propio directorio `openwebui_pipelines`, sincronizado con el directorio de pipelines de dentro del contenedor, y todos los scripts de *Python* se alojan en el directorio `python-scripts`, separando el módulo de ingestión de datos en `openciti`, el motor de RAG en `rag`, y finalmente, el programa principal que pone en marcha el motor, `main.py`.

3.2.1. Consumo y vectorización de OpenCTI

3.2.1.1. Ingestión

El cometido de este módulo es conectarse a OpenCTI mediante el SDK (provisto por el paquete oficial *pycti*), configurando los credenciales adecuados (*IP* de *OpenCTI* y clave de la API). El cliente es pesado, y para evitar ser cargado múltiples veces, por ejemplo, por diferentes hilos de ejecución, ralentizando el sistema completo, sigue un patrón *Singleton*.

Para cada tipo de entidad CTI que se cargará (concretamente *threat actors*, *malware*, *attack patterns*, *indicators* y *reports*), el módulo define qué atributos necesita recuperar. Para todos los tipos de objeto se cargan identificador, nombre, descripción y fecha de creación. Para *Threat Actors* y *Reports*, estos son los únicos campos recuperados. Se

recuperan campos adicionales para:

- **Malware:** Para este grupo se recupera también el campo `malware_types`, que define a qué categorías de malware pertenece (e.g. *ransomware*, *trojan*, *backdoor*)
- **Attack Patterns:** Para este grupo se recupera además el campo `x_mitre_id`, que es el identificador MITRE asociado a una técnica cuando el *Attack Pattern* pertenece al modelo MITRE ATT&CK[11]
- **Indicators:** Se cargan adicionalmente los campos `pattern_type` y `pattern`, que indican la sintaxis en la que está escrito el campo `pattern`, y la expresión técnica que define el *IoC*. Esto es, el valor y la forma asociado al indicador (por ejemplo, un dominio cuyo valor es *officeonline.com*).

Así, no se descarga todo el objeto CTI, ya que esto sería un proceso muy largo y pesado, sino sólo los aspectos relevantes para las fases posteriores del procesado.

Los datos se recuperan mediante ingesta paginada, consumiendo todas las páginas devueltas por *OpenCTI* hasta que no quedan más disponibles. Esto se debe a que *OpenCTI* puede tener del orden de cientos de miles o incluso millones de entidades, por lo que una única llamada a la API no sería suficiente para recuperar todos los datos o, según la configuración de *timeouts*, podría no devolver nada.

Una vez finalizada la extracción, guarda los datos obtenidos en ficheros JSON independientes. Por ejemplo, `threat_actors.json` o `indicators.json`. En este caso, se guardan en el directorio `/opt/data/raw/opencti`. Como referencia, una ejecución de este programa, concretamente el 8 de junio de 2026, carga 55 *threat actors*, 355 *malwares*, 410 *attack patterns*, 515569 *indicators* y 5924 *reports*.

Todo este proceso lo lleva a cabo el script `opencti/ingest.py`.

3.2.1.2. Normalización

Este modulo se encarga de la normalización de los datos brutos extraídos de *OpenCTI*. Transforma los objetos recuperados en documentos textuales homogéneos adecuados para su posterior vectorización. Los datos extraídos de *OpenCTI* se almacenan como JSON bruto, por lo que cada objeto recuperado mantiene su estructura original. Esa representación no es la más adecuada para realizar búsquedas semánticas, ya que los modelos de lenguajes trabajan mejor sobre texto descriptivo que sobre JSON complejos.

Es una capa intermedia entre la ingesta bruta y la generación de *embeddings* para contener, para cada objeto CTI, la información relevante en un formato legible, y más importante, igual para todos los objetos de un mismo tipo para facilitar la búsqueda por parte del LLM al vectorizar el texto. Aunque indicadores y reportes se dejan tal y como están, se incluyen en esta parte la búsqueda de relaciones de cierto tipos de objetos en *OpenCTI* para mejorar la semántica asociada al mismo:

- **Threat Actors:** se incluyen relaciones tipo `uses` para recuperar *Attack Patterns* usados y *Malware* asociado.
- **Malware:** se incluyen también relaciones tipo `uses` para recuperar *Attack Patterns* usados.

- **Attack Patterns:** se incluyen relaciones tipo **uses** para recuperar *Threat Actors* que usan estos patrones y técnicas.

El resultado final del proceso de normalización son documentos estructurados, uniformes para cada tipo de objeto recuperado de *OpenCTI*. Por razones de procesado y eficiencia, se guardarán en formato JSONL (*JSON Lines*), que almacena un objeto JSON por línea, aunque extrayendo el contenido y dejándolo en un formto humanamente legible, quedaría como los ejemplo en el Listado 1.

Esta parte del proceso es llevada a cabo por los scripts **normalizer**, que define los procesos de normalización para cada tipo de objeto y **normalize_all.py**, que lee los ficheros JSON guardados, procesa la normalización de cada uno de estos ficheros y los guarda en su correspondiente fichero JSONL.

```

1 # Threat Actor
2
3 THREAT_ACTOR: <nombre_del_actor>
4 Descripción: <descripción_del_actor>
5
6 TTPs usados:
7 - <TTP_1> (<ID_MITRE_1>)
8 - <TTP_2> (<ID_MITRE_2>)
9 - <TTP_N> (<ID_MITRE_N>)
10
11 Malware asociado:
12 - <malware_1>
13 - <malware_2>
14 - <malware_N>
15
16 Creado: <fecha_de_creación>
17
18
19 # Malware
20
21 MALWARE: <nombre_del_malware>
22 Tipos: <tipo_1>, <tipo_2>, <tipo_N>
23 Descripción: <descripción_del_malware>
24
25 TTPs usados:
26 - <TTP_1> (<ID_MITRE_1>)
27 - <TTP_2> (<ID_MITRE_2>)
28 - <TTP_N> (<ID_MITRE_N>)
29
30 Creado: <fecha_de_creación>
31
32
33 # Attack Pattern / TTP
34
35 TTP: <nombre_de_la_técnica>
36 ID_MITRE: <identificador_MITRE>
37 Descripción: <descripción_de_la_técnica>
38
39 Usado por:
40 - <actor_1>
41 - <actor_2>
42 - <actor_N>
43

```

```

44 Creado: <fecha_de_creación>
45
46
47 # Indicator / IOC
48
49 INDICATOR (IOC)
50 Patrón: <patrón_del_indicador>
51 Tipo: <tipo_de_patrón>
52 Creado: <fecha_de_creación>
53 Válido desde: <fecha_de_inicio_de_validez>
54
55
56 # Report
57
58 REPORT: <nombre_del_report>
59 Descripción: <descripción_del_report>
60 Creado: <fecha_de_creación>

```

Listado 1: Ejemplos de estructuras normalizadas

3.2.1.3. Generación de *embeddings* e inserción en *ChromaDB*

Tras la fase de normalización, los objetos CTI extraídos desde OpenCTI se encuentran representados como documentos textuales semiestructurados. Aunque estos documentos ya son legibles y contienen información relevante sobre actores, malware, técnicas, indicadores e informes, todavía no pueden ser utilizados directamente para realizar búsquedas semánticas. Para ello es necesario transformarlos en representaciones vectoriales, conocidas como *embeddings*, que permiten comparar documentos y consultas en función de su similitud semántica.

El proceso de vectorización consiste en cargar los ficheros `.jsonl` generados durante la normalización, extraer el campo `text` de cada documento y convertirlo en un vector numérico mediante el modelo `sentence-transformers/all-MiniLM-L6-v2`. Este modelo genera una representación densa del significado del texto, de forma que documentos conceptualmente similares quedan próximos dentro del espacio vectorial. En paralelo, también se cargan los ficheros de metadatos asociados a cada colección, comprobando que el número de textos y metadatos coincide antes de generar los embeddings. Esta comprobación es importante para garantizar que cada vector queda asociado al documento y metadatos correctos.

Los embeddings generados se almacenan en ficheros `.npy`, separados por tipo de entidad. Así, se generan ficheros independientes para `threat_actors`, `malware`, `attack_patterns`, `reports` e `indicators`. Esta separación permite mantener una organización coherente del corpus CTI y facilita la carga posterior en colecciones diferenciadas dentro de ChromaDB. Además, guardar los embeddings como ficheros NumPy permite reutilizarlos sin necesidad de recalcularlos cada vez que se quiera reconstruir la base vectorial.

Una vez generados los embeddings, el siguiente paso consiste en cargarlos en ChromaDB. Para ello, el script de inserción se conecta al servidor de ChromaDB mediante `HttpClient`. Para cada tipo de entidad, el script crea o recupera una colección con el mismo nombre de la entidad, por ejemplo `threat_actors`, `malware`, `attack_patterns`, `reports` o `indicators`. Las colecciones se configuran utilizando similitud coseno, una métrica habitual para comparar embeddings en sistemas de recuperación semántica.

Antes de insertar los documentos en ChromaDB, el script valida que los textos normalizados, los metadatos y los embeddings tengan la misma longitud. Esta validación evita desalineaciones entre documentos, vectores y metadatos. Posteriormente, la inserción se realiza por lotes, asociando a cada documento su identificador, su texto, su embedding y sus metadatos. De este modo, cada entrada almacenada en ChromaDB conserva tanto la representación vectorial necesaria para la búsqueda semántica como la información textual y de trazabilidad necesaria para interpretar el resultado recuperado.

El resultado final de este proceso es una base vectorial organizada en colecciones CTI, donde cada documento procedente de OpenCTI puede ser recuperado por similitud semántica. Durante una consulta, el backend RAG genera un embedding de la pregunta o del indicador detectado y consulta las colecciones de ChromaDB para obtener los documentos más cercanos. Estos documentos se incorporan al contexto final como información contextual o inferida, diferenciándola de la inteligencia explícita recuperada directamente desde OpenCTI.

La vectorización y carga en ChromaDB son, por tanto, una parte esencial del sistema RAG. Esta fase permite complementar la información confirmada de OpenCTI con contexto semántico adicional, como actores, malware, técnicas, informes o indicadores relacionados conceptualmente con la consulta. Sin embargo, la similitud vectorial no implica una relación confirmada entre entidades CTI. Por este motivo, los documentos recuperados desde ChromaDB se tratan como contexto inferido y se separan explícitamente de la información considerada confirmada.

Este proceso lo llevan a cabo los scripts *embed_all_st.py*, para la generación de *embeddings*, y *chroma_load.py*

3.2.2. Frontend, backend y motor de RAG

3.2.2.1. OpenWebUI

3.2.2.2. Servidor *Pipelines*

3.2.2.3. Backend y motor RAG

```

/opt
├── chat-feats
│   ├── docker-compose.yml
│   └── README.md
├── modelos_ia
│   ├── all-MiniLM-L6-v2
│   ├── cyberpal
│   │   └── Modelfile
│   ├── lily
│   │   └── Modelfile
├── ollama
├── openwebui_data
├── openwebui_pipelines
│   ├── cybeross_pipeline
│   ├── lily_pipeline
│   ├── rag_pipeline
│   ├── cybeross_pipeline.py
│   └── lily_pipeline.py
├── python-scripts
│   ├── opencti
│   │   ├── chroma_load.py
│   │   ├── client.py
│   │   ├── embed_all_st.py
│   │   ├── ingest.py
│   │   ├── normalize_all.py
│   │   ├── normalizer.py
│   │   └── rag_client.py
│   ├── rag
│   │   ├── context
│   │   │   ├── builder.py
│   │   │   └── context_profiles.py
│   │   ├── app.py
│   │   ├── ioc_context_builder.py
│   │   ├── ioc_resolver.py
│   │   └── query_classifier.py
│   ├── main.py
│   ├── README.md
│   └── requirements.txt

```

Figura 3: Estructura de directorios del proyecto

4. Evaluación y discusión de resultados

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	1	2
¿Muestra bien el estado y confidence?	1	2
Menciona y separa el contexto inferido	0	2
¿Separa relaciones fuertes y débiles?	1	2
¿Alucina?	1	1
¿Es útil para análisis?	1	2
Puntuación	5	11

Tabla 1: Resultados para dominio conocido

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	1	2
¿Muestra bien el estado y confidence?	1	2
Menciona y separa el contexto inferido	2	2
¿Separa relaciones fuertes y débiles?	1	2
¿Alucina?	1	1
¿Es útil para análisis?	1	2
Puntuación	6	11

Tabla 2: Resultados para dirección IP conocida

5. Conclusiones y trabajo futuro

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	2	2
¿Muestra bien el estado y confidence?	2	2
Menciona y separa el contexto inferido	0	2
¿Separa relaciones fuertes y débiles?	1	2
¿Alucina?	2	1
¿Es útil para análisis?	1	2
Puntuación	8	11

Tabla 3: Resultados para hash conocido

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	0	0
¿Muestra bien el estado y confidence?	0	0
Menciona y separa el contexto inferido	0	0
¿Separa relaciones fuertes y débiles?	0	0
¿Alucina?	0	0
¿Es útil para análisis?	0	0
Puntuación	0	0

Tabla 4: Resultados para pregunta por contexto inferido

Anexos

Anexo A. Organización del trabajo.

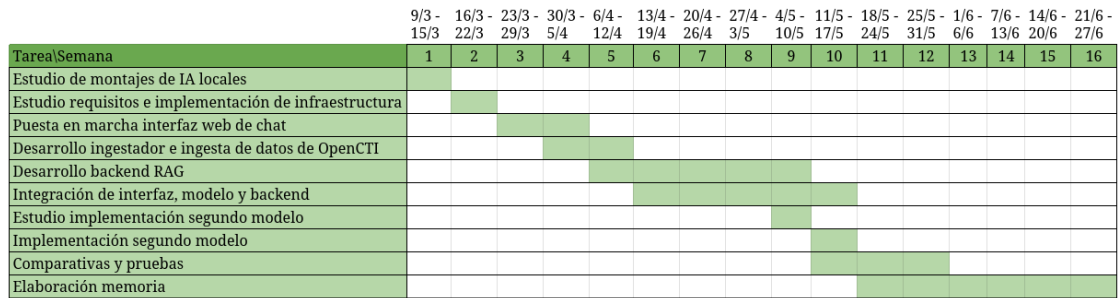


Figura 4: Diagrama de Gantt

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	1	2
¿Muestra bien el estado y confidence?	0	2
Menciona y separa el contexto inferido	1	2
¿Separa relaciones fuertes y débiles?	0	2
¿Alucina?	0	2
¿Es útil para análisis?	0	2
Puntuación	2	12

Tabla 5: Resultados de relación con actores

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	2	2
¿Muestra bien el estado y confidence?	1	2
Menciona y separa el contexto inferido	2	2
¿Separa relaciones fuertes y débiles?	2	1
¿Alucina?	2	1
¿Es útil para análisis?	2	2
Puntuación	12	10

Tabla 6: Resultados de separación fuerte/débil explícita

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	2	2
¿Muestra bien el estado y confidence?	2	2
Menciona y separa el contexto inferido	2	2
¿Separa relaciones fuertes y débiles?	2	2
¿Alucina?	2	1
¿Es útil para análisis?	2	2
Puntuación	11	11

Tabla 7: Resultados de pregunta abierta

	Lily	CyberOSS
Promedio	6,29	9,43
Sobre 10	5,24	7,86

Tabla 8: Resultados finales

Tarea	Horas
Estudio de montajes de IA locales	10
Estudio requisitos e implementación de infraestructura	10
Puesta en marcha interfaz web de chat	20
Desarrollo ingestador e ingesta de datos	25
Desarrollo backend y motor RAG	55
Integración interfaz, modelo y backend	70
Estudio de segundo modelo	5
Implementación segundo modelo	5
Comparativas y pruebas	25
Desarrollo memoria	
Total	225

Tabla 9: Tabla de tareas y tiempos

Anexo B. Ficheros desarrollados.

```
1 # ingest.py
2 import os
3 import json
4 from dotenv import load_dotenv
5 from .client import ApiClient
6
7 load_dotenv()
8
9 DATA_ROOT=os.getenv("DATA_ROOT", "/opt/data")
10 RAW_DIR=f"{DATA_ROOT}/raw/opencti"
11 os.makedirs(RAW_DIR, exist_ok=True)
12
13 class OpenCTIIngestor:
14     def __init__(self):
15         self.api = ApiClient().api
16
17     def _save_json(self, filename, data):
18         path = os.path.join(RAW_DIR, filename)
19
20         with open(path, 'w', encoding="utf-8") as f:
21             json.dump(data, f, indent=2, ensure_ascii=False)
22
23         print(f"[] Guardado en {path} ({len(data)} objetos)")
24
25     def _ingest_entities(self, list_function, label, first=200,
26 custom_attributes=None):
27         """
28         Método general para ingerir entidades desde OpenCTI usando
29         paginación oficial:
30         - withPagination=True
31         - pagination.hasNextPage / endCursor
32         - data["entities"] devuelto por pycti
33         """
34         print(f"\n[+] Iniciando ingesta de {label}...")
35
36         all_entities = []
37         data = {"pagination": {"hasNextPage": True, "endCursor": None
38 }}
39
40         while data["pagination"]["hasNextPage"]:
41             after = data["pagination"]["endCursor"]
42
43             # Llamada oficial según documentación OpenCTI
44             data = list_function(
45                 first=first,
46                 after=after,
47                 customAttributes=custom_attributes,
48                 withPagination=True,
49                 orderBy="created_at",
50                 orderMode="asc",
51             )
52
53             if not data or "entities" not in data:
54                 print(f"[] Error obteniendo datos para {label}")
55                 break
```

```

54         batch = data["entities"]
55         all_entities.extend(batch)
56
57         print(f"        - Descargados {len(batch)} nuevos ({len(
all_entities)} total)")
58
59         print(f"[] Ingesta completada: {label} ({len(all_entities)}
objetos)")
60         return all_entities
61
62
63
64     def ingest_threat_actors(self):
65         custom = """
66             id
67             name
68             description
69             created
70         """
71         data = self._ingest_entities(self.api.threat_actor.list, "
Threat Actors", 200, custom)
72
73         self._save_json("threat_actors.json", data)
74         return data
75
76     def ingest_malware(self):
77         custom = """
78             id
79             name
80             malware_types
81             description
82             created
83         """
84         data = self._ingest_entities(self.api.malware.list, "Malware",
200, custom)
85
86         self._save_json("malware.json", data)
87         return data
88
89     def ingest_attack_patterns(self):
90         custom = """
91             id
92             name
93             x_mitre_id
94             description
95             created
96         """
97         data = self._ingest_entities(self.api.attack_pattern.list, "
Attack Patterns", 200, custom)
98
99         self._save_json("attack_patterns.json", data)
100         return data
101
102     def ingest_indicators(self):
103         custom = """
104             id
105             pattern
106             pattern_type

```

```

107         created
108         valid_from
109     """
110     data = self._ingest_entities(self.api.indicator.list, "
Indicators (IOCs)", 200, custom)
111
112     self._save_json("indicators.json", data)
113     return data
114
115
116 def ingest_reports(self):
117     custom = """
118         id
119         name
120         description
121         created
122     """
123     data = self._ingest_entities(self.api.report.list, "Reports",
100, custom)
124
125     self._save_json("reports.json", data)
126     return data
127
128
129 if __name__ == "__main__":
130     ingestor = OpenCTIIngestor()
131
132     actors = ingestor.ingest_threat_actors()
133     malware = ingestor.ingest_malware()
134     ttps = ingestor.ingest_attack_patterns()
135     iocs = ingestor.ingest_indicators()
136     reports = ingestor.ingest_reports()
137
138     print("\n=== RESUMEN FINAL ===")
139     print(f" Threat Actors: {len(actors)}")
140     print(f" Malware: {len(malware)}")
141     print(f" TTPs: {len(ttps)}")
142     print(f" IOCs: {len(iocs)}")
143     print(f" Reports: {len(reports)}")

```

Listado 2: ingest.py

```

1 # normalizer.py
2 from .ingest import OpenCTIIngestor
3 from .client import ApiClient
4 from dotenv import load_dotenv
5
6 load_dotenv()
7
8
9 class OpenCTINormalizer:
10     def __init__(self):
11         self.api = ApiClient().api
12
13     def _get_relationships(self, entity_id, relationship_type=None):
14         relationships = self.api.stix_core_relationship.list(
15             fromId = entity_id,
16             relationship_type = relationship_type,
17             first=2000

```

```

18         )
19         return relationships or []
20
21     def _join_list(self, items):
22         return ", ".join(items) if items else "N/A"
23
24     # Normalizadores individuales de cada tipo de objeto
25     def normalize_threat_actor(self, actor):
26         uses_ttps = self._get_relationships(actor["id"], "uses")
27         uses_malware = [
28             r for r in uses_ttps if r.get("to", {}).get("entity_type")
29             == "Malware"
30         ]
31
32         uses_attack_patterns = [
33             r for r in uses_ttps if r.get("to", {}).get("entity_type")
34             == "Attack-Pattern"
35         ]
36
37         ttps = [
38             f"{r['to']['name']} ({r['to'].get('x_mitre_id', '')})"
39             for r in uses_attack_patterns if r.get("to")
40         ]
41
42         malware = [
43             r["to"]["name"] for r in uses_malware if r.get("to")
44         ]
45
46         text = f"""
47         THREAT_ACTOR: {actor.get("name", "N/A")}
48         Descripción: {actor.get("description", "N/A")}
49
50         TTPs usados:
51         - {chr(10).join(ttps) if ttps else "N/A"}
52
53         Malware asociado:
54         - {chr(10).join(malware) if malware else "N/A"}
55
56         Creado: {actor.get("created", "N/A")}
57         """
58         return text.strip()
59
60     def normalize_malware(self, malware):
61         relationships = self._get_relationships(malware["id"], "uses")
62
63         ttps = [
64             f"{r['to']['name']} ({r['to'].get('x_mitre_id', '')})"
65             for r in relationships if r.get("to", {}).get("entity_type")
66             == "Attack-Pattern"
67         ]
68
69         text = f"""
70         MALWARE: {malware.get("name")}
71         Tipos: {self._join_list(malware.get("malware_types", []))}
72         Descripción: {malware.get("description", "N/A")}
73
74         TTPs usados:

```

```

73 - {chr(10).join(ttps) if ttps else "N/A"}
74
75 Creado: {malware.get("created")}
76 """
77
78     return text.strip()
79
80
81     def normalize_attack_pattern(self, ttp):
82
83         used_by = self._get_relationships(tp["id"], "uses")
84         actors = [
85             r["from"]["name"] for r in used_by
86             if r.get("from", {}).get("entity_type") == "Threat-Actor"
87         ]
88
89         text = f"""
90 TTP: {ttp.get("name")}
91 ID_MITRE: {ttp.get("x_mitre_id")}
92 Descripción: {ttp.get("description")}
93
94 Usado por:
95 - {chr(10).join(actors) if actors else "N/A"}
96
97 Creado: {ttp.get("created")}
98 """
99
100     return text.strip()
101
102
103     def normalize_indicator(self, ioc):
104         return f"""
105 INDICATOR (IOC)
106 Patrón: {ioc.get("pattern")}
107 Tipo: {ioc.get("pattern_type")}
108 Creado: {ioc.get("created")}
109 Válido desde: {ioc.get("valid_from")}
110 """.strip()
111
112
113     def normalize_report(self, report):
114         return f"""
115 REPORT: {report.get("name")}
116 Descripción: {report.get("description", "N/A")}
117 Creado: {report.get("created")}
118 """.strip()
119
120
121     def normalize_all_threat_actors(self, actors):
122         return [self.normalize_threat_actor(a) for a in actors]
123
124     def normalize_all_malware(self, malware):
125         return [self.normalize_malware(m) for m in malware]
126
127     def normalize_all_ttps(self, ttps):
128         return [self.normalize_attack_pattern(t) for t in ttps]
129
130     def normalize_all_reports(self, reports):

```

```

131         return [self.normalize_report(r) for r in reports]
132
133     def normalize_all_iocs(self, iocs):
134         return [self.normalize_indicator(i) for i in iocs]
135
136
137     def metadata_threat_actor(self, actor):
138         return {
139             "id": actor.get("id"),
140             "name": actor.get("name"),
141             "type": "Threat Actor"
142         }
143
144     def metadata_malware(self, malware):
145         return {
146             "id": malware.get("id"),
147             "name": malware.get("name"),
148             "type": "Malware"
149         }
150
151     def metadata_ttp(self, ttp):
152         return {
153             "id": ttp.get("id"),
154             "name": ttp.get("name"),
155             "type": "Attack-Pattern"
156         }
157
158     def metadata_report(self, report):
159         return {
160             "id": report.get("id"),
161             "name": report.get("name"),
162             "type": "Report"
163         }
164
165     def metadata_ioc(self, ioc):
166         return {
167             "id": ioc.get("id"),
168             "type": "Indicator"
169         }

```

Listado 3: normalizer.py

```

1  # normalize_all.py
2  import os
3  import json
4  from dotenv import load_dotenv
5
6  from .normalizer import OpenCTINormalizer
7
8  load_dotenv()
9
10 DATA_ROOT = os.getenv("DATA_ROOT", "/opt/data")
11
12 RAW_DIR = f"{DATA_ROOT}/raw/opencti"
13 NORMALIZED_DIR = f"{DATA_ROOT}/normalized"
14
15 os.makedirs(NORMALIZED_DIR, exist_ok=True)
16
17

```

```

18 def load_json(filename):
19     path = os.path.join(RAW_DIR, filename)
20     if not os.path.exists(path):
21         print(f"[!] No existe el fichero: {path}")
22         return []
23     with open(path, "r", encoding="utf-8") as f:
24         return json.load(f)
25
26 def save_normalized_jsonl(filename, texts):
27     path = os.path.join(NORMALIZED_DIR, filename)
28     with open(path, "w", encoding="utf-8") as f:
29         for t in texts:
30             f.write(json.dumps({"text": t}) + "\n")
31         print(f"[ ] Guardado JSONL: {path} ({len(texts)} documentos)")
32
33 def save_metadata(filename, metas):
34     path = os.path.join(NORMALIZED_DIR, filename)
35     with open(path, "w", encoding="utf-8") as f:
36         for m in metas:
37             f.write(json.dumps(m) + "\n")
38     print(f"[ ] Guardado JSONL: {path} ({len(metas)} metadatos)")
39
40 if __name__ == "__main__":
41     normalizer = OpenCTINormalizer()
42
43     print("=== NORMALIZANDO ENTIDADES ===\n")
44
45     actors_raw = load_json("threat_actors.json")
46     actors_text = normalizer.normalize_all_threat_actors(actors_raw)
47     actors_meta = [normalizer.metadata_threat_actor(a) for a in
48 actors_raw]
49
50     save_normalized_jsonl("threat_actors.jsonl", actors_text)
51     save_metadata("threat_actors_meta.jsonl", actors_meta)
52
53     malware_raw = load_json("malware.json")
54     malware_text = normalizer.normalize_all_malware(malware_raw)
55     malware_meta = [normalizer.metadata_malware(m) for m in
56 malware_raw]
57
58     save_normalized_jsonl("malware.jsonl", malware_text)
59     save_metadata("malware_meta.jsonl", malware_meta)
60
61     ttps_raw = load_json("attack_patterns.json")
62     ttps_text = normalizer.normalize_all_ttps(ttps_raw)
63     ttps_meta = [normalizer.metadata_ttp(t) for t in ttps_raw]
64
65     save_normalized_jsonl("attack_patterns.jsonl", ttps_text)
66     save_metadata("attack_patterns_meta.jsonl", ttps_meta)
67
68
69     iocs_raw = load_json("indicators.json")
70     iocs_text = normalizer.normalize_all_iocs(iocs_raw)
71     iocs_meta = [normalizer.metadata_ioc(i) for i in iocs_raw]
72
73     save_normalized_jsonl("indicators.jsonl", iocs_text)

```



```

74     save_metadata("indicators_meta.jsonl", iocs_meta)
75
76
77     reports_raw = load_json("reports.json")
78     reports_text = normalizer.normalize_all_reports(reports_raw)
79     reports_meta = [normalizer.metadata_report(r) for r in reports_raw
80 ]
81
82     save_normalized_jsonl("reports.jsonl", reports_text)
83     save_metadata("reports_meta.jsonl", reports_meta)
84
85     print("== NORMALIZACIÓN COMPLETADA ==\n")

```

Listado 4: normalize_all.py

```

1  # embed_all_st.py
2  import os
3  import json
4  import numpy as np
5  from sentence_transformers import SentenceTransformer
6
7  DATA_ROOT = os.getenv("DATA_ROOT", "/opt/data")
8
9  NORMALIZED_DIR = f"{DATA_ROOT}/normalized"
10 EMB_DIR = f"{DATA_ROOT}/embeddings"
11
12 os.makedirs(EMB_DIR, exist_ok=True)
13
14 def load_texts_jsonl(filename):
15     path = os.path.join(NORMALIZED_DIR, filename)
16     texts = []
17     with open(path, "r", encoding="utf-8") as f:
18         for line in f:
19             texts.append(json.loads(line)["text"])
20     return texts
21
22
23 def load_metadata(filename):
24     path = os.path.join(NORMALIZED_DIR, filename)
25     metas = []
26     with open(path, "r", encoding="utf-8") as f:
27         for line in f:
28             metas.append(json.loads(line))
29     return metas
30
31 def save_embeddings(filename, vecs):
32     path = os.path.join(EMB_DIR, filename)
33     np.save(path, vecs)
34     print(f"[] Guardado embeddings: {path} shape={vecs.shape}")
35
36 def check_alignment(texts, metas, entity_type):
37     if len(texts) != len(metas):
38         raise ValueError(
39             f"[ERROR] Desalineación en {entity_type}: {len(texts)}
40             textos vs {len(metas)} metadatos"
41         )
42     print(f"[] Alineación OK para {entity_type}: {len(texts)} items")
43

```

```

44 if __name__ == "__main__":
45     print("[+] Cargando modelo 'sentence-transformers/all-MiniLM-L6-v2
...")
46     model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-
v2")
47
48     # Threat Actors
49     actors_text = load_texts_jsonl("threat_actors.jsonl")
50     actors_meta = load_metadata("threat_actors_meta.jsonl")
51     check_alignment(actors_text, actors_meta, "Threat Actors")
52
53     emb = model.encode(actors_text, batch_size=64, show_progress_bar=
True)
54     save_embeddings("threat_actors.npy", emb)
55
56
57     # Malware
58     malware_text = load_texts_jsonl("malware.jsonl")
59     malware_meta = load_metadata("malware_meta.jsonl")
60     check_alignment(malware_text, malware_meta, "Malware")
61
62     emb = model.encode(malware_text, batch_size=64, show_progress_bar=
True)
63     save_embeddings("malware.npy", emb)
64
65
66     # Attack Patterns
67     ttps_text = load_texts_jsonl("attack_patterns.jsonl")
68     ttps_meta = load_metadata("attack_patterns_meta.jsonl")
69     check_alignment(ttps_text, ttps_meta, "TTPs")
70
71     emb = model.encode(ttps_text, batch_size=64, show_progress_bar=
True)
72     save_embeddings("attack_patterns.npy", emb)
73
74
75     # Reports
76     reports_text = load_texts_jsonl("reports.jsonl")
77     reports_meta = load_metadata("reports_meta.jsonl")
78     check_alignment(reports_text, reports_meta, "Reports")
79
80     emb = model.encode(reports_text, batch_size=64, show_progress_bar=
True)
81     save_embeddings("reports.npy", emb)
82
83
84     # Indicators
85     iocs_text = load_texts_jsonl("indicators.jsonl")
86     iocs_meta = load_metadata("indicators_meta.jsonl")
87     check_alignment(iocs_text, iocs_meta, "IOCs")
88
89     emb = model.encode(iocs_text, batch_size=256, show_progress_bar=
True)
90     save_embeddings("indicators.npy", emb)

```

Listado 5: embed_all_st.py

```

1 # chroma_load.py
2 import os

```

```

3 import json
4 import numpy as np
5 from chromadb import HttpClient
6 from chromadb.config import Settings
7
8 DATA_ROOT = os.getenv("DATA_ROOT", "/opt/data")
9
10 NORMALIZED_DIR = f"{DATA_ROOT}/normalized"
11 EMB_DIR = f"{DATA_ROOT}/embeddings"
12
13 CHROMA_HOST = os.getenv("CHROMADB_HOST", "localhost")
14 CHROMA_PORT = int(os.getenv("CHROMADB_PORT", "8000"))
15
16 chroma = HttpClient (
17     host=CHROMA_HOST,
18     port=CHROMA_PORT
19 )
20
21 def load_texts_jsonl(filename):
22     path = os.path.join(NORMALIZED_DIR, filename)
23     texts = []
24     with open(path, "r", encoding="utf-8") as f:
25         for line in f:
26             texts.append(json.loads(line)["text"])
27     return texts
28
29 def load_metadata(filename):
30     path = os.path.join(NORMALIZED_DIR, filename)
31     meta = []
32     with open(path, "r", encoding="utf-8") as f:
33         for line in f:
34             meta.append(json.loads(line))
35     return meta
36
37 def load_embeddings(filename):
38     path = os.path.join(EMB_DIR, filename)
39     return np.load(path)
40
41
42 def create_or_get_collection(name):
43     try:
44         return chroma.get_collection(name=name)
45     except:
46         return chroma.create_collection(
47             name=name,
48             metadata={"hnsw:space": "cosine"}
49         )
50
51 def insert_into_chroma(collection, embeddings, documents, metadata,
52     batch=1000):
53     print(f"[+] Insertando documento en lotes de {batch}...")
54     total = len(documents)
55
56     for i in range(0, total, batch):
57         end = i + batch
58         batch_embeddings = embeddings[i:end].tolist()
59         batch_docs = documents[i:end]
60         batch_meta = metadata[i:end]

```

```

60     batch_ids = [m["id"] for m in batch_meta]
61
62     collection.add(
63         ids=batch_ids,
64         embeddings=batch_embeddings,
65         documents=batch_docs,
66         metadatas=batch_meta
67     )
68
69     print(f"        {end}/{total} insertados")
70
71
72 if __name__ == "__main__":
73     print("=== CARGA A CHROMADB ===")
74
75     tipos = ["threat_actors", "malware", "attack_patterns", "reports",
76             "indicators"]
77
78     for t in tipos:
79         print(f"\n=== {t.upper()} ===")
80
81         collection = create_or_get_collection(t)
82
83         texts = load_texts_jsonl(f"{t}.jsonl")
84         meta = load_metadata(f"{t}_meta.jsonl")
85         emb = load_embeddings(f"{t}.npy")
86
87         if len(texts) != len(meta) or len(texts) != emb.shape[0]:
88             raise ValueError(
89                 f"[ERROR] Desalineación en {t}: {len(texts)}
90                 textos, {len(meta)} metadatos, {emb.shape[0]} embeddings"
91             )
92
93         insert_into_chroma(
94             collection,
95             embeddings=emb,
96             documents=texts,
97             metadata=meta,
98             batch = 1000 if t != "indicators" else 5000
99         )
100
101     print(f"[] {t} cargados.\n")

```

Listado 6: chroma_load.py

Bibliografía

- [1] Davidjbianco. *Enterprise Detection & Response: The Pyramid of Pain*. Enterprise Detection & Response. 1 de mar. de 2013. URL: <https://detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html> (visitado 27-05-2026).
- [2] OpenCTI Team. *OpenCTI Data Model*. OpenCTI Data Model. URL: <https://docs.opencti.io/latest/usage/data-model/> (visitado 27-05-2026).
- [3] *Modelo extenso de lenguaje*. En: *Wikipedia, la enciclopedia libre*. 25 de mayo de 2026. URL: https://es.wikipedia.org/w/index.php?title=Modelo_extenso_de_lenguaje&oldid=173636604 (visitado 27-05-2026).
- [4] Cole Stryker. *¿Qué son los grandes modelos de lenguaje (LLM)?* — IBM. 6 de oct. de 2021. URL: <https://www.ibm.com/es-es/think/topics/large-language-models> (visitado 27-05-2026).
- [5] *¿Qué es un LLM? - Explicación de los modelos de lenguaje grandes - AWS*. Amazon Web Services, Inc. URL: <https://aws.amazon.com/es/what-is/large-language-model/> (visitado 27-05-2026).
- [6] Woosuk Kwon et al. «Efficient Memory Management for Large Language Model Serving with PagedAttention». En: *Proceedings of the 29th Symposium on Operating Systems Principles*. SOSP '23: 29th Symposium on Operating Systems Principles. Koblenz Germany: ACM, 23 de oct. de 2023, págs. 611-626. ISBN: 979-8-4007-0229-7. DOI: 10.1145/3600006.3613165. URL: <https://dl.acm.org/doi/10.1145/3600006.3613165> (visitado 29-05-2026).
- [7] *Segolilylabs/Lily-Cybersecurity-7B-v0.2 · Hugging Face*. URL: <https://huggingface.co/segolilylabs/Lily-Cybersecurity-7B-v0.2> (visitado 29-05-2026).
- [8] Matan Levi et al. *Toward Cybersecurity-Expert Small Language Models*. Ver. 1. 2025. DOI: 10.48550/ARXIV.2510.14113. URL: <https://arxiv.org/abs/2510.14113> (visitado 29-05-2026). Prepublicado.
- [9] OpenWebUI. *Open WebUI*. URL: <https://docs.openwebui.com/> (visitado 01-06-2026).
- [10] OpenWebUI. *Pipelines / Open WebUI*. URL: <https://openwebui.com/features/extensibility/pipelines/> (visitado 01-06-2026).
- [11] MITRE. *MITRE ATT&CK®*. URL: <https://attack.mitre.org/> (visitado 08-06-2026).